Tab 1

- SQL databases: postgresql, mysql, sqlite, sql server, oracle, cockroach db
  They all have common language which is SQL with very little nuances unlike NOSQL databases which have nothing in common except the fact that it doesn't use sql

- NoSQL database: mongo db, redis, elasticsearch, firebase, dynamo db
  These are mostly specialized db, redis is used for cache/in-memory, elasticsearch is used for search/aggregations, firebase is good for real time & mobile apps, dynamo db and mongo db are mostly general purpose

- SELECT * FROM users // selects all columns/field
- CREATE TABLE people(id INTEGER, handle TEXT, name TEXT, age INTEGER, balance INTEGER, is_admin BOOLEAN) // creates a table named people
- ALTER TABLE employees RENAME TO contractors
- ALTER TABLE contractors RENAME COLUMN salary TO invoice;
- ALTER TABLE contractors ADD COLUMN job_title TEXT
- ALTER TABLE contractors DROP COLUMN is_manager

- Available data types are INT, REAL (this is for floating point numbers), TEXT (also usually databases like mysql and postgres support VARCHAR), NULL, BLOB (this is for binary data).

- Constraints:
  CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT NOT NULL, age INTEGER NOT NULL, username TEXT UNIQUE, is_admin BOOLEAN)

  PRIMARY KEY can only be used on one field in a table, and it is NOT NULL and UNIQUE

  CREATE TABLE departments(id INT PRIMARY KEY, department_name TEXT NOT NULL)

  CREATE TABLE employees (id INT PRIMARY KEY, name TEXT NOT NULL, department_id INTEGER, CONSTRAINT fk_departments FOREIGN KEY (department_id) REFERENCES departments(id)

  Note: here fk_departments is the name which we gave for this constraint which can be anything

  FOREIGN KEY is a key in a table that references the id in another table.

- CRUD:

  Create, read, update, delete

  SELECT * FROM crud

  INSERT INTO employees (id, name, title) VALUES (1, “Gautham”, “Engineer”)

- AUTO INCREMENT: usually the field with PRIMARY KEY will auto increment, it avoids making a select request in order to know the id and then insert a new one with the incremented id, it takes care of it.
- SELECT count(*) FROM users

- WHERE CLAUSE:

  SELECT name FROM users WHERE power_level >= 9000
  SELECT name FROM users WHERE first_name IS NULL
  SELECT name FROM users WHERE first_name IS NOT NULL

- DELETE:
  DELETE FROM employees WHERE id = 251
  Note: run a select statement first in order check and verify what you are deleting

- DataBase Backups:
  Usually snapshots of databases are taken on a daily or hourly basis and kept in storage and if something bad happens then we can restore from the snapshots, but we got some problem here which is still we miss some data which was inserted into db after the snapshot was taken and before the table was deleted. These snapshots are deleted after some time, likely monthly or so, as it would be expensive to store all snapshots forever.

  Append only log:
  Here we maintain the logs i.e., sql statements in storage, and if we unfortunately delete the db then we can rerun the logs and restore the db.

  Soft Delete:
  Here we never delete from db, instead we maintain something like another field and mark it as deleted. We actually don't run any DELETE query. It can be a primary concern and it can be expensive and also our queries might become slow.

- UPDATE:
  UPDATE employees SET job_title = “Software Engineer”, salary = 1500000 WHERE id = 251

- ORM (Object Relational Mapping):
  It is typically a library
  The primary benefit an ORM provides is that it maps your database records to in-memory objects.
  When using an ORM, you call methods and functions made available via the ORM’s API.
  We don’t write raw SQL queries.

- AS keyword used for aliases
  It doesn’t persist in db, it is for time-being, for sql query
  SELECT employee_id AS id, employee_name AS name FROM employees

- IIF function is nothing but a ternary
  SELECT quantity, IIF(quantity < 10, “Order more”, “in stock”) AS directive FROM

products

- BETWEEN clause:
  SELECT employee_name, salary FROM employees WHERE salary BETWEEN 100000 AND 200000
  SELECT employee_name, salary FROM employees WHERE salary NOT BETWEEN 100000 AND 200000

- DISTINCT: Removes duplicate records from the resulting query
  SELECT DISTINCT previous_company FROM employees

- SELECT employee_name, salary FROM employees WHERE employee_name = “Gautham” AND salary BETWEEN 100000 AND 200000
  SELECT employee_name, salary FROM employees WHERE employee_name = “Gautham” OR salary BETWEEN 100000 AND 200000
  SELECT employee_name, salary FROM employees WHERE employee_name IN (“Gautham”, “Gokulakonda”)

- % operator matches zero or more characters
  SELECT * FROM products WHERE product_name LIKE ‘banana%’
  SELECT * FROM products WHERE product_name LIKE ‘%banana’
  SELECT * FROM products WHERE product_name LIKE ‘%banana%

- _ operator matches exactly one character
  SELECT * FROM products WHERE product_name LIKE ‘_oot

- LIMIT:
  SELECT * FROM products WHERE product_name LIKE ‘_oot’ LIMIT 50

- ORDER BY:
  SELECT * FROM products ORDER BY price
  SELECT * FROM products ORDER BY price DESC

- ORDER BY clause must come first before LIMIT
  SELECT * FROM products ORDER BY price LIMIT 5

- Aggregations:
  SELECT count(*) FROM users
  SELECT sum(salary) FROM employees
  SELECT max(price) FROM products
  SELECT min(price) FROM products
  SELECT album_id, count(song_id) FROM songs GROUP BY album_id
  SELECT song_name, avg(song_length) FROM songs GROUP BY song_name

- HAVING clause:
  When we want to filter the results of a GROUP BY clause even further, we can use the HAVING clause. The HAVING clause specified the condition for a group.
  The HAVING clause is similar to the WHERE clause, but it operates in groups after they have been grouped, rather than rows before they have been grouped.
  It works with aggregations like count, sum etc.

SELECT album_id, name, count(song_id) AS count FROM songs WHERE name LIKE '%play' GROUP BY album_id HAVING count > 5 ORDER BY album_id DESC

- SELECT song_name, round(avg(song_length), 1) FROM songs

- SUB QUERIES:
- SELECT id, song_name, artist_id FROM songs WHERE artist_id IN (SELECT id FROM artists WHERE artist_name LIKE 'RICK%')
  SELECT * FROM transactions WHERE user_id = (SELECT id FROM users WHERE name = 'david')

- SQL is a full programming language. We usually use it to interact with data stored in tables. But it's quite flexible and powerful
  SELECT 5 + 10 AS sum
  SELECT * FROM users WHERE age_in_days > (40*365)

- One to one relationship : simple, here mostly all items will be on same table
- One to many relationship: Here we will maintain 2 tables, and in one table we maintain foreign key which references another table. Ex: users table and tweets table
- Many to many relationship: Here we usually have 3 tables, the third table is typically a joining table,
  Ex: users and groups. Typically there are two things in table and they are foreign keys of the respective tables and one constraint is rows must be identical
  Joining table:
  CREATE TABLE product_suppliers (product_id INTEGER, supplier_id INTEGER, UNIQUE(product_id, supplier_id))

- More normalization means table will have less redundancy and more accuracy and improve data integrity i.e., data is valid always
  1st Normal Form:
  Every row must have a unique primary key
  There can be no nested tables
  2nd Normal Form: (should first follow 1st normal form)
  All columns that are not part of the primary key must only be dependent on the entire primary key
  3rd Normal Form: (should follow both 1 and 2)
  All the columns not in the primary key are dependent only on the primary key
  Boyce Codd Normal Form:
  A column that is part of the primary key may not be dependent on a column that is not part of the primary key

- JOINS:
  It allows us to query multiple tables at the same time
  SELECT * FROM employees INNER JOIN departments ON employees.department_id =

departments.id // Here we get fields from both the tables

SELECT students.name, classes.name FROM students INNER JOIN classes ON classes.class_id = students.class_id

SELECT students.name, classes.name FROM students s INNER JOIN classes c ON c.class_id = s.class_id

SELECT users.name, sum(transactions.amount) AS sum, count(transactions.id) AS count FROM users LEFT JOIN transactions ON users.id = transactions.user_id GROUP BY users.id ORDER BY sum DESC

We can retrieve the same records with a right join and left join by flipping the position of the tables in the join statement.
Joins basically work on records not fields.
A FULL JOIN returns all of the records in both the tables.

- Performance:
  SQL INDEXES: An index is an in-memory structure that ensures that the queries we run on a database are performant. Most database indexes are binary trees! The binary tree is stored in ram instead of on disk and it makes it easy to look up the location of an entire row.
  Primary key columns are indexed by default, ensuring you can look up a row by its id very quickly. However, if you have other columns that you want to be able to do quick lookups on, you'll need to index them.

  CREATE INDEX index_name ON table_name (column_name)

  While indexes make specific kinds of lookups much faster, they also add performance overhead - they can slow down a database in other ways. Think about it, if you index every column, you could have hundreds of binary trees in memory! That needlessly bloats the memory usage of your database. It also means that each time you insert a record, that record needs to be added to many trees - slowing down your insert speed

  A binary trees makes lookups O(log(n))

  Add indexes to columns that you frequently perform lookups on (usually here the lookup means the field which we mention in the where clause)

  Multi column index:
  CREATE INDEX first_name_last_name_age_idx ON users (first_name, last_name, age)
  A multi column index is sorted by the first column first, the second column next and so forth. A lookup on only the first column in a multi-column index gets almost all of the performance improvements that it would get from its own single-column index. However lookups on only the second or third column will have very degraded performance

Tab 2

Points to remember:

1. In SQL, the != (or <>) operator does not compare NULL values because NULL represents an unknown value. In SQL, any comparison with NULL results in NULL (which is treated as unknown or false in a WHERE clause).

If referee_id is NULL, the condition becomes NULL != 2, which results in NULL.
Since SQL ignores NULL in WHERE conditions (it is neither TRUE nor FALSE), rows where referee_id is NULL are not included in the result.

To handle NULL values in comparisons, use IS NULL or IS NOT NULL explicitly.

```
select name

from customer

where referee_id != 2 or referee_id is null;
```

2. Compare the dates of w1 and w2 using the DATEDIFF() function to check if they are consecutive days (with a difference of 1 day).

```
select w1.id from Weather w1 inner join Weather w2 on datediff(w1.recordDate, w2.recordDate) = 1 where w1.temperature > w2.temperature
```

```
SELECT W1.id

FROM Weather W1

JOIN Weather W2

   ON W1.recordDate = DATE_ADD(W2.recordDate, INTERVAL 1 DAY)

WHERE W1.temperature > W2.temperature;
```

3. [sql date and time functions]

```
 select date_format(trans_date, "%Y-%m") as month, country, count(amount) as trans_count, sum(if(state="approved",1,0)) as approved_count, sum(amount) as trans_total_amount, sum(if(state="approved", amount,0)) as approved_total_amount from Transactions group by country, month
```